Staff Management System

A digital leave, permission & travel approval platform built for St. John's University of Tanzania's multi-level institutional workflow

ST. JOHN'S UNIVERSITY OF TANZANIA

COURSE / UNIT
[Course Code – Unit Name]

GROUP MEMBERS
[Name 1]
[Name 2]
[Name 3]

PRESENTED TO
[Instructor Name]

DATE
[Presentation Date]

Presentation Agenda

Six evaluation areas, covered in order

1	**Problem Identification & Relevance** Why this system, why now	10 pts
2	**Requirements Analysis** Functional & non-functional	10 pts
3	**System Design** Architecture & approval flow	15 pts
4	**Implementation & Functionality** What actually works, live	20 pts
5	**User Interface & UX** Design system & usability	10 pts
6	**Database Design & Management** Schema, integrity, security	10 pts

Problem Identification & Relevance

Clear problem statement, justification, objectives, relevance of the proposed solution

The Problem

Leave, permission, and travel requests at SJUT pass through **multiple approval levels** — Unit Head, HOD, Dean, DVC, HR — historically tracked on paper or over email.

Staff have **no visibility** into where their request currently sits in the chain.

No institutional **audit record** of who approved what, and when.

No structured way to **delegate** to an acting/holding staff member when the requester is away.

Objectives

- Digitize the full multi-level approval chain for Leave, Permission, and Travel requests.
- Give **every registered staff member** — any role, any department — a working self-service portal, not just a curated few.
- Provide a complete, chronological **audit trail** of every action in the system.
- Support acting-staff **delegation**, including HOD override-on-behalf when needed.

Relevance: the design directly mirrors SJUT's real organizational hierarchy and was validated end-to-end with real staff accounts across real departments — not a synthetic demo dataset. The same approval-chain pattern generalizes to any institution with layered sign-off requirements.

Requirements Analysis

Functional & non-functional requirements, user needs, feasibility

Functional Requirements

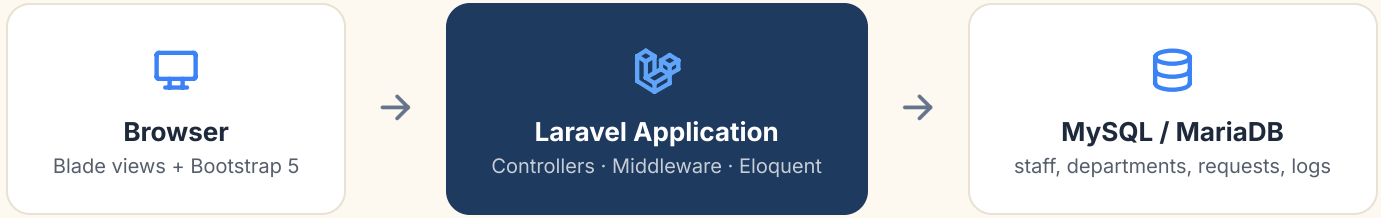
- Submit Leave, Permission, or Travel requests
- Route through the correct multi-stage chain per request type
- Acting-staff delegation — accept, decline, or HOD override
- Role-based approval queues and dashboards
- In-app notifications (bell dropdown + full page)
- Full audit trail: login, logout, password changes, views, every approval action
- Live search across every major list page

Non-Functional Requirements

- **Multi-tenant isolation** — every query is university-scoped, so data never crosses institutions
- **Server-side validation** — e.g. acting staff must be in the same department, enforced beyond the UI
- **Role-gated access** — single login, panel access decided server-side per role
- **Usability** — consistent search pattern, responsive layout, compact professional density
- **Auditability** — every state-changing action is logged automatically via middleware

System Design

Architecture, approval-flow design, and the logical system structure



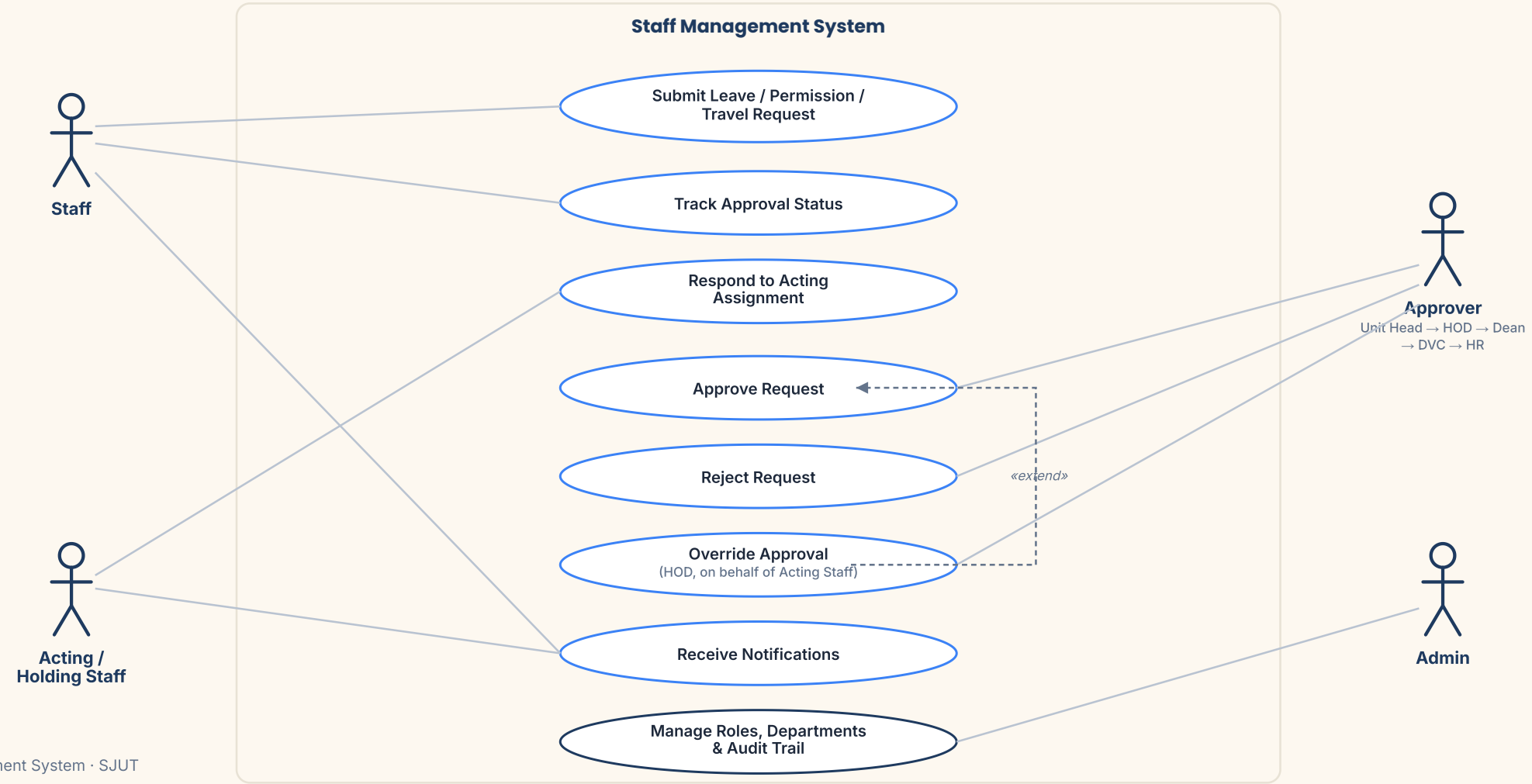
Leave Approval Pipeline



Two-layer role design: a Spatie role (e.g. “hod”) controls what a user can *see*; a dedicated foreign key on the Department/School record (`hod_staff_id`, `dean_staff_id`) decides who can actually *approve* for that unit — keeping permission labels separate from operational routing authority.

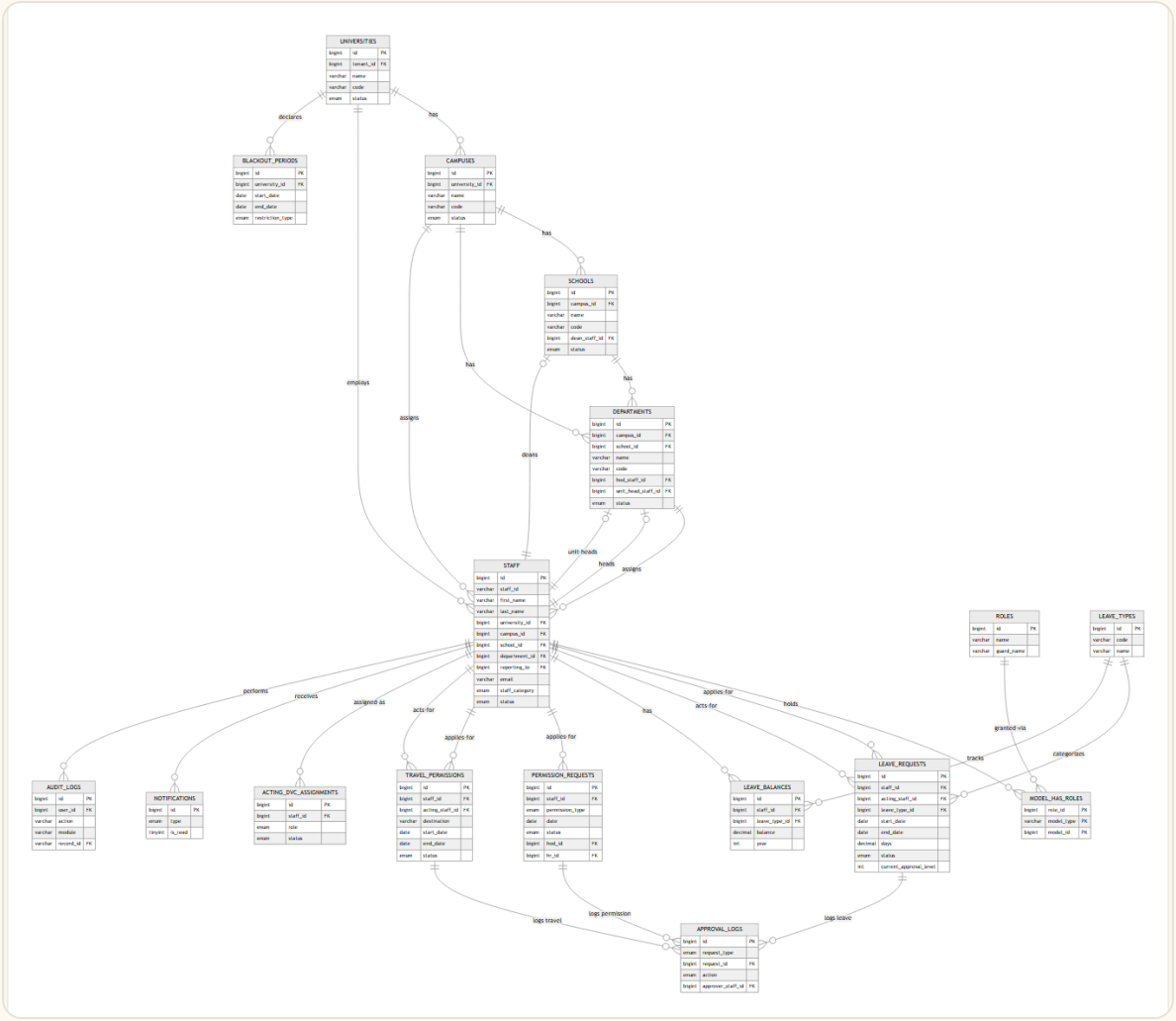
Use Case Diagram (UML)

Actors and the core interactions they have with the system



Entity-Relationship Diagram

Institute hierarchy, staff/roles, and the three request workflows with their shared approval-log and notification trail



Implementation & Functionality

Correctness, completeness, feature integration — what actually works, live

PHP 8.4 | Laravel | MySQL / MariaDB | Blade | Bootstrap 5 | Spatie Permission | Tabler Icons

Full request lifecycle
Leave, Permission & Travel — submit through final sign-off, live status at every stage

Acting-staff delegation
Accept / decline flow, plus HOD override-on-behalf with correct timeline reflection

Live search everywhere it matters
Leave, Permission, and Role Management pages — debounced, no page reload needed on most

Comprehensive audit trail
Login, logout, password changes, page views, and every create/approve/reject action

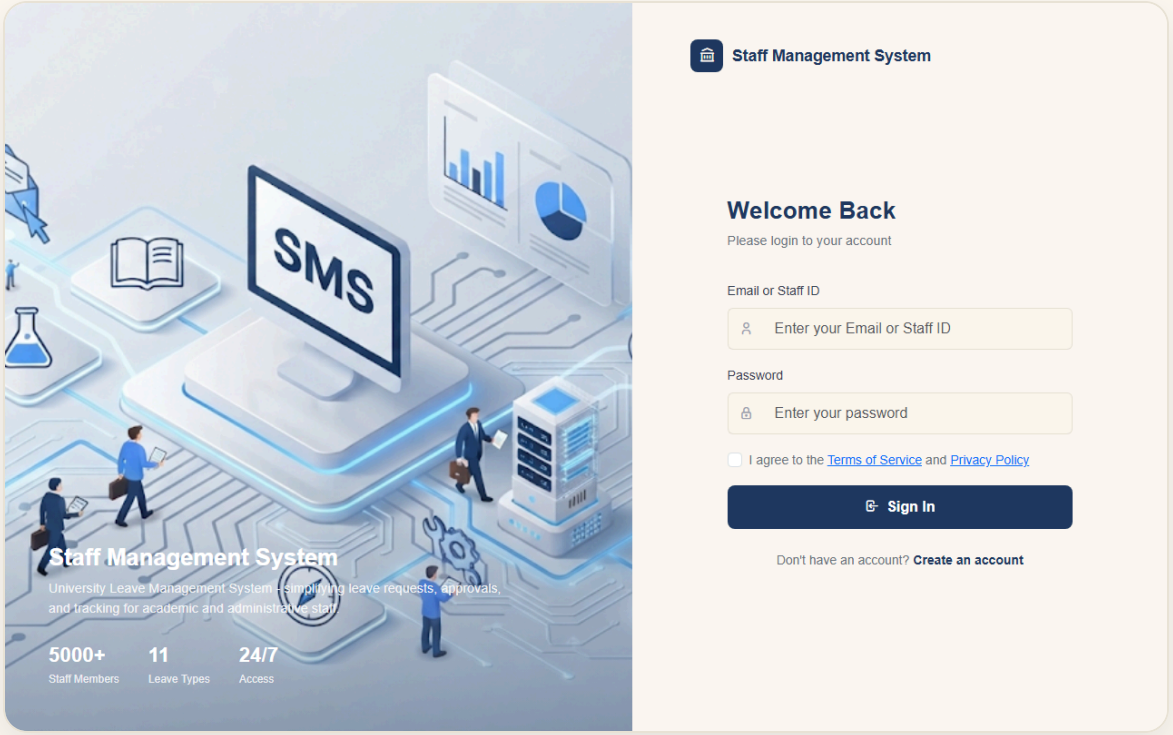
In-app notification system
Real-time bell dropdown + dedicated notifications page with read/unread filtering

Admin management tools
Roles, departments, universities, data cleanup — all with working search

Verified, not assumed: the entire approval chain was tested end-to-end using real SJUT staff accounts spanning Unit Head, HOD, Dean, DVCA, and HR — not synthetic placeholder data.

User Interface & User Experience

Usability, navigation, consistency, accessibility, visual presentation

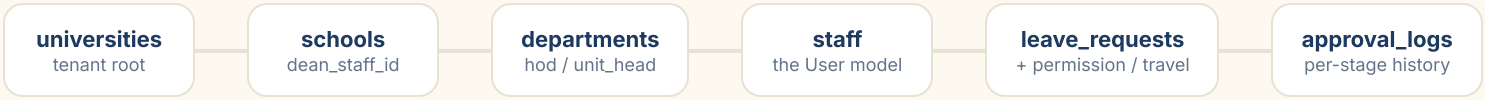


Design System

- **Single login, role-gated** — one entry point, server decides what each role can reach
- **Consistent search pattern** — the same live-search component reused across Leave, Permission, and Role Management
- **Uniform tab navigation** — Notifications filters now match the Profile page's tab style
- **Clear unread/read states** — tinted background, accent border, bold title for anything needing attention
- **Responsive layout** — mobile sidebar toggle and breakpoints down to phone width

Database Design & Management

Structure, relationships, normalization, data integrity and security



Normalization & Separation of Concerns

- `approval_logs` (per-request approval history) is kept separate from `audit_logs` (system-wide activity) and `hold_position_history` (acting-staff specific) — each log serves one purpose
- Leave, Permission, and Travel each own their approval-state columns rather than sharing one generic table
- `staff` carries university/campus/department foreign keys, enforcing the institutional hierarchy at the data layer

Integrity & Security

- Cross-tenant isolation enforced in every query touching multi-university data
- Referential rules validated at the application layer (e.g. `Rule::exists` scoped by department)
- Full CRUD + auth audit trail via a catch-all logging middleware, immune to any single controller forgetting to log

Where Each Requirement Was Met

A one-glance map from the evaluation rubric to what you just saw

#	EVALUATION AREA	COVERED BY	MARKS
1	Problem Identification & Relevance	Slide 3 — SJUT approval-chain pain points & objectives	10
2	Requirements Analysis	Slide 4 — functional & non-functional breakdown	10
3	System Design	Slides 5–7 — architecture, UML use-case diagram, full ERD	15
4	Implementation & Functionality	Slide 8 + live demo — verified with real accounts	20
5	User Interface & UX	Slide 9 — design system & consistency	10
6	Database Design & Management	Slide 10 — schema, normalization, integrity	10
Group Total			**75**

Thank You

Questions & Live Demonstration